Lazy: API decisions and tradeoffs

Decision record for Java architects — updated 2 October 2026 — branch `prototype/lazy-one-class`.

This document records the design developed in this discussion and embodied in the specification of `java.lang.util.Lazy` in this branch. It summarizes decisions and consequences; the source Javadoc remains the normative contract for this prototype. “Adopted” means adopted in this branch, not approved for the Java platform or committed to a release.

The starting point was the work associated with JDK-8336664, including `LazyConstant` and the `List.ofLazy`, `Map.ofLazy`, and `Set.ofLazy` factories, followed by Brian Goetz's email reproduced in the discussion. The email supplied the overall direction. Defects in the subsequent naming, typing, failure, memory, and collection decisions described here should be blamed on Per only. The pros and cons below are architectural assessments, not quotations or evidence of wider consensus.

The architectural direction

The adopted design is one non-generic, final utility class, `java.lang.util.Lazy`, with two factory families. Cache factories own storage and return familiar `Supplier`, `Function`, `List`, `Map`, or `Set` interfaces. Updater factories use a caller-owned field or array element and return a functional-interface implementation whose method locates that storage. Both families defer computation, but their storage obligations and available policies differ. Factory names end in `Cache` or `Updater` so callers can see which contract applies before reading an overload.

Two principles from Brian's proposal drive the design. First, laziness has several legitimate meanings: initialization with irreversible side effects needs different coordination from a repeatable calculation. Second, ordinary users should not need to understand VM stability machinery to use the API correctly. Constant folding remains an optimization opportunity underneath a usable library abstraction.

One class gives laziness a single discovery point and lets the specification explain the common principle before separating the two storage models. The cost is a larger class whose users must still distinguish a cache's owned storage from an updater's reserved-default-value protocol. The new `java.lang.util` package is neutral with respect to the former `java.util` and `java.lang.invoke` homes, but introduces another exported `java.base` package and requires migration from both old class names. The design covers selected useful combinations rather than every combination of coordinates, storage, failure policy, and memory ordering.

The cache family offers two modes, a default `COMPUTE_ONCE` scalar-supplier factory, and acquire/release publication. The updater family offers three modes, including `PLAIN`; its general field factory takes explicit lookup authority, while narrower static- and instance-field convenience factories derive authority from their caller. A direct-array convenience factory needs no lookup. The distinct `CacheMode` and `UpdaterMode` enums prevent passing an updater-only policy to a cache factory. The main costs are mode qualification on general factories, possible duplicate work or permanent failure depending on the family, and extra obligations for caller-owned storage.

How the design evolved

Stage	Step taken	Step not taken
API organization	Followed Brian's one-class direction through `Lazy`, then temporarily separated `java.util.LazyCaches` and `java.lang.invoke.LazyUpdaters`; this branch reunites both families in `java.lang.util.Lazy`.	Did not expose the earlier family of `OfAbsolute`, `OfRelative`, `OfIndexed`, and `OfIndexedRelative` interfaces, or keep two public facades as aliases.

Stage	Step taken	Step not taken
Factory usability	Replaced `BoundFunction` with `Function`, used `ofScalar` for both scalar cache shapes, then renamed the general factories to `ofScalarCache`, `ofListCache`, `ofMapCache`, `ofSetCache`, `ofLocationUpdater`, or `ofArrayUpdater`; added `ofStaticFieldUpdater`, `ofFieldUpdater`, and a direct-array `ofArrayUpdater` convenience for common reference-valued shapes.	Did not retain bare `of`, `ofContext`, or factory names that hide the storage family. The two `ofScalarCache` overload shapes still admit some method-reference ambiguity.
Policy naming	Evolved the proposal's `InitMode` and the branch's `ONCE` / `RACE` terminology into `Lazy.CacheMode.COMPUTE_ONCE` / `PUBLISH_ONCE`; renamed updater `CAS` / `SYNCHRONIZED` to `PUBLISH_ONCE` / `SERIALIZED` to describe their guarantees.	Did not make the public mode names depend on a particular atomic instruction or locking mechanism, or offer independent retry, exclusion, and fencing switches.
Mode typing	Renamed the former independent `LazyCaches.Mode` and `LazyUpdaters.Mode` enums to `Lazy.CacheMode` and `Lazy.UpdaterMode`.	Did not merge the enums merely because both contain `PUBLISH_ONCE`, or expose updater-only modes to cache factories.
Updater mode selection	Required `UpdaterMode` as the first argument of the general updater factories and listed `PUBLISH_ONCE` first as the usual choice for repeatable computations; the three narrow conveniences fix that mode.	Did not restore implicit-`PUBLISH_ONCE` overloads for arbitrary field and array shapes; those calls still state their policy.
Memory contract	Replaced the earlier volatile-access guarantee with acquire/release publication, including recorded failures.	Did not promise a total ordering across different lazy values or extra synchronization from repeated access.
Field access	Explored a named-field `Lazy.ofField`, moved caller-owned storage to `LazyUpdaters.ofLocation`, and now exposes it as `Lazy.ofLocationUpdater`. That factory takes a string field selector and checks the field's type against the functional-interface result. `Lazy.ofStaticFieldUpdater` and `Lazy.ofFieldUpdater` cover static and instance fields with `Supplier` and `Function`, respectively, using caller-sensitive access.	Did not bypass access checks or keep the later bytecode-cracked accessor-lambda scheme. A convenience cannot access a private field outside its caller's authority.
Array access	Extended updaters from one- and two-index access to any positive index count supported by the array rank and method arity. `ofLocationUpdater` selects an instance or static array field by name; `ofArrayUpdater` takes the array itself, with no selector string or caller lookup. Added a reference-array, one-index convenience using `ObjIntFunction`; multiple index coordinates and primitive arrays use the general factory.	Did not add a cache factory that returns an indexed-array accessor, infer arbitrary array expressions from lambdas, or promise index permutation.
Computation timing	Retained declaration-site factories that store one computation in the updater, and added use-site overloads that take a computation as the updater's first argument on each access.	Did not require a callback stored by the updater when different accesses need different computations. The caller must keep computations interchangeable for a shared location.

Stage	Step taken	Step not taken
Contract completion	Added the set cache (originally `ofSet`) and specified collection views, entries, traversal failures, streams, callback retention, composition, and execution context.	Did not treat ordinary interface defaults as sufficient to define these interactions.
Implementation	After the API reviews, brought implementations into line and added focused and general collection regressions.	Did not remove the earlier lazy APIs, settle their migration policy, or establish performance claims from benchmarks in this work.

Brian's final sketch used `BoundSupplier` and `InitMode`, still overloaded `of`, and did not include a set cache. The current design follows that direction with substantive later refinements.

Current API surface

The following is a signature synopsis of `java.lang.util.Lazy`; bodies, annotations, and Javadoc are omitted. The `Cache` and `Updater` suffixes identify whether the factory provides storage or uses caller-owned storage. The two mode types make that distinction checkable by the compiler.

```
public enum CacheMode { COMPUTE_ONCE, PUBLISH_ONCE }
public enum UpdaterMode { PUBLISH_ONCE, SERIALIZED, PLAIN }
public static final class InitializationError extends Error { ... }
public interface ObjIntFunction<T, R> {
    R apply(T t, int value);
}

public static <T> Supplier<T> ofScalarCache(
        Supplier<? extends T> computingFunction);
public static <T> Supplier<T> ofScalarCache(
        CacheMode mode, Supplier<? extends T> computingFunction);

public static <C, T> Function<C, T> ofScalarCache(
        CacheMode mode, Function<? super C, ? extends T> computingFunction);

public static <T> List<T> ofListCache(
        CacheMode mode, int size, IntFunction<? extends T> computingFunction);
public static <K, V> Map<K, V> ofMapCache(
        CacheMode mode, Set<? extends K> keys,
        Function<? super K, ? extends V> computingFunction);
public static <E> Set<E> ofSetCache(
        CacheMode mode, Set<? extends E> elementCandidates,
        Predicate<? super E> computingFunction);

public static <T> Supplier<T> ofStaticFieldUpdater(
        Class<?> owner, String name, Supplier<T> computing);
public static <R, T> Function<R, T> ofFieldUpdater(
        Class<R> owner, String name, Function<R, T> computing);
public static <T> T ofLocationUpdater(
        UpdaterMode mode, MethodHandles.Lookup lookup, Class<T> type,
        String fieldSelector, T computingFunction);
public static <T> T ofLocationUpdater(
        UpdaterMode mode, MethodHandles.Lookup lookup, Class<T> type,
        String fieldSelector);
public static <T> T ofArrayUpdater(
        UpdaterMode mode, Class<T> type, T computingFunction);
public static <T> T ofArrayUpdater(
        UpdaterMode mode, Class<T> type);
public static <T> ObjIntFunction<T[], T> ofArrayUpdater(
        Class<T> elementType, ObjIntFunction<T[], T> computing);
```

`InitializationError` has a public `InitializationError(String message)` constructor and pertains only to `CacheMode.COMPUTE_ONCE`. Nine ready-made use-site accessor interfaces are nested in `Lazy` and described below. `ObjIntFunction<T,R>` is the one-reference-plus-one-`int` counterpart to

`java.util.function.ObjIntConsumer<T>`; its reference input is `T`, its result is `R`, and its method follows `Function.apply` naming. The nested `StaticObjectFieldSupplier extends Supplier<Object>` provides the concrete functional-interface token used by `ofStaticFieldUpdater`.

Factory	Unit of retained state	Access	Policy selection
`ofScalarCache(Supplier)` / `ofScalarCache(CacheMode, Supplier)`	One value per returned supplier	`get()`	Explicit mode, or `COMPUTE_ONCE` by default
`ofScalarCache(CacheMode, Function)`	One value per returned function, across all contexts	`apply(context)`	Explicit mode
`ofListCache`	One value per index	`get(index)` and ordinary list operations	One explicit mode, applied independently per element
`ofMapCache`	One value per copied key	`get(key)` and ordinary map operations	One explicit mode, applied independently per mapped value
`ofSetCache`	One Boolean membership result per copied candidate	`contains(candidate)` and ordinary set operations	One explicit mode, applied independently per candidate
`ofStaticFieldUpdater`	One caller-owned, caller-accessible static mutable field	Returned `Supplier<T>.get()`	`PUBLISH_ONCE` fixed by the convenience factory
`ofFieldUpdater`	One caller-owned, caller-accessible instance field per receiver	Returned `Function<R,T>.apply(receiver)`	`PUBLISH_ONCE` fixed by the convenience factory
`ofLocationUpdater`	Caller-owned named field or array element reached through a named field	Returned interface's functional method	Explicit `UpdaterMode`
`ofArrayUpdater(UpdaterMode, Class, ...)`	Caller-owned direct-array element	Returned interface's functional method	Explicit `UpdaterMode`
`ofArrayUpdater(Class<T>, ObjIntFunction<T[],T>)`	One element per index of each supplied reference array	Returned `ObjIntFunction<T[],T>.apply(array, index)`	`PUBLISH_ONCE` fixed by the convenience factory

Among cache factories, only `ofScalarCache(Supplier)` supplies a default mode. There is no field, segment, or direct-array **cache** factory and no indexed-context cache shape. A list cache provides indexed access together with size, views, iteration, and streams; the price is a collection contract rather than a minimal coordinate interface. This is a surface-area choice, not a demonstrated claim of identical performance to an array. `Lazy.ofLocationUpdater` instead updates an element reached through a named array field, while `Lazy.ofArrayUpdater` updates an element of an array supplied directly to the updater.

Consolidation and migration

This branch replaces the two preview facades rather than preserving forwarding aliases. The internal implementations remain separated by storage mechanism; the public class, package, naming, and specification become one. Clients migrate as follows:

Previous API	API in this branch
`java.util.LazyCaches`	`java.lang.util.Lazy`
`LazyCaches.Mode`	`Lazy.CacheMode`
`LazyCaches.InitializationError`	`Lazy.InitializationError`
`LazyCaches.ofScalar` / `ofList` / `ofMap` / `ofSet`	`Lazy.ofScalarCache` / `ofListCache` / `ofMapCache` / `ofSetCache`
`java.lang.invoke.LazyUpdaters`	`java.lang.util.Lazy`

Previous API	API in this branch
`LazyUpdaters.Mode`	`Lazy.UpdaterMode`
`LazyUpdaters.ofLocation` / `ofArray`	`Lazy.ofLocationUpdater` / `ofArrayUpdater`
`LazyUpdaters` ready-made accessor interfaces	The same nested interface names under `Lazy`

The suffixes cost characters but make the storage owner visible in calls such as `Lazy.ofListCache(...)` and `Lazy.ofLocationUpdater(...)`. This is useful because a returned updater and a returned cache can both have a `get`-like method while imposing different duties on the caller. A single class makes the families easier to discover and compare, but its specification is longer and its mode types remain separate. `java.lang.util` gives the merged class one neutral home; the cost is a new exported package and source and binary incompatibility with both previous facades. Clients must import `java.lang.util.Lazy` explicitly; automatic `java.lang` imports do not include subpackages. No claim of compatibility with code compiled against those names is made for this prototype branch. The pre-existing `LazyConstant` and `List.ofLazy`, `Map.ofLazy`, and `Set.ofLazy` APIs are outside this facade move. The new `@since 29` tags are provisional while this prototype builds on a feature-version-28 tree; release metadata must be aligned with the chosen integration release.

Standard interfaces, overload resolution, and inference

Returning `Supplier` and `Function` avoids introducing new access interfaces. Replacing `BoundFunction<C,T>.get(C)` with `Function<C,T>.apply(C)` improves interoperability and gives users ordinary composition. The corresponding cost is semantic: `Function` alone does not tell the reader where storage lives or whether distinct arguments receive distinct results.

The `ofScalarCache(CacheMode, Function)` overload owns **one slot**, not a cache indexed by context. For example:

```
Function<String, Integer> length =
        Lazy.ofScalarCache(Lazy.CacheMode.PUBLISH_ONCE, String::length);

length.apply("abc");    // Installs 3.
length.apply("hello");  // Returns the retained 3.
```

An instance-derived property normally needs its own `ofScalarCache(CacheMode, Function)` holder per instance. A static holder shared across instances would share one value. Supplying the context at access time permits a noncapturing callback; it does not eliminate the holder or guarantee allocation-free lambda execution. A null context is rejected even after initialization.

The shared `ofScalarCache` name makes the scalar category visible but again overloads supplier-shaped and context-shaped computations. Explicit zero-argument and one-argument lambdas generally distinguish them; overloaded or inexact method references—such as references to constructors with several arities—can make overload selection ambiguous. A destination result type does not generally rescue that selection. The earlier `ofContext` name avoided this ambiguity and made the access shape explicit, at the cost of another method name.

The remaining `var` issue is ordinary inference: Java cannot infer `String` merely from `s -> s.length()` when no context type is otherwise supplied. A typed destination, an explicitly typed lambda, a suitable method reference, or explicit type arguments supplies that information. For example:

```
var length = Lazy.ofScalarCache(Lazy.CacheMode.PUBLISH_ONCE,
                                (String s) -> s.length());
```

An additional `Class<C>` argument for the context-shaped overload was explored as an inference aid but was not adopted. It would make `var` examples easier to type, while adding an argument and another factory form despite there being no field type to validate in this case. This is a considered alternative, not a permanent rejection of any future convenience overload.

Composition retains the ordinary `Function` ordering: `compose` runs the preceding function on every application, and `andThen` runs the following function after every successful lazy access. Their results are not automatically cached. Composition itself records no additional initialization failure; nested lazy accesses still follow their own failure rules. This avoids silently changing familiar composition semantics, but requires callers to place expensive work inside the intended lazy boundary.

Two coordinated policies, named for their guarantees

Dimension	`COMPUTE_ONCE`	`PUBLISH_ONCE`
Intended use	Initialization, including side effects that must not repeat	Memoization where repeated computation is acceptable
Invocation count per slot	At most once; zero if never needed	Competing invocations and retries after failure are permitted
Competing access	Waits for the one invocation	May compute another candidate
Successful access	Returns the retained result	Returns the installed winner, including when its own candidate loses
Failure	Permanent	Propagated to that caller; not retained
Principal benefit	Prevents duplicate initialization and repeated side effects	Avoids requiring all callers to wait for a designated initializing computation
Principal cost	Waiting, permanent failure, and possible dependency deadlock	Duplicate work, discarded results, and side effects that may repeat

`COMPUTE_ONCE` states what is limited; `PUBLISH_ONCE` states what is committed. They are more informative than `ONCE` / `RACE` and avoid encoding a particular instruction such as CAS. “Once” does not require eager execution. The installed result is consistent across successful accesses to one slot, but the identity or contents of the winning candidate need not be predictable across executions.

Initialization versus memoization is an intended-use distinction, not a static-versus-instance restriction. Either mode can serve either location where the chosen factory supports it.

The two cache policies bundle exclusion and failure handling. The benefit is a small choice set; the cost is that a cache cannot request, for example, serialized initialization with retry or concurrent computation with sticky failure. `PUBLISH_ONCE` is not a promise that the API or computing function is lock-free or wait-free. Discarded results are not closed or otherwise disposed of by `Lazy`.

Mode types and the caller-owned-storage updater

`Lazy.CacheMode` and `Lazy.UpdaterMode` are distinct nested enums. `CacheMode` offers `COMPUTE_ONCE` and `PUBLISH_ONCE`; `UpdaterMode` offers `PUBLISH_ONCE`, `SERIALIZED`, and `PLAIN` for caller-owned field and array-element storage. The same spelling `PUBLISH_ONCE` describes the shared successful-publication outcome, but the two families have different storage and failure protocols. A value from either enum cannot be passed to a factory expecting the other. Both retain enum conveniences such as `values()` and `switch` expressions; separate static imports of the two `PUBLISH_ONCE` constants would be ambiguous, so examples qualify the mode.

The updater's `PUBLISH_ONCE` replaces the earlier name `CAS`: it names the one installed value rather than the operation used to install it. `SERIALIZED` replaces `SYNCHRONIZED`: it promises one computation at a time per storage location without requiring a particular locking implementation. Unlike `Lazy.CacheMode.COMPUTE_ONCE`, `SERIALIZED` allows another access to retry after failure.

The `ofLocationUpdater` factories identify an instance or static field by string. An instance-field selector is the field's simple name, with its owner inferred from the updater interface's concrete receiver type; a static-field selector is the owner's binary name followed by a dot and the field name. One or more `int` parameters following the receiver, or comprising all storage parameters for a static array field, select an array element through that field. The general `ofArrayUpdater` factories need no selector: their first storage parameter is the array and the following one or more

`int` parameters select an element. In a use-site interface, the computing-function parameter comes before these storage parameters. The index count must not exceed the array rank and remains subject to ordinary Java and JVM method-parameter limits. The interface fixes that count: `int...` erases to one `int[]` parameter and cannot stand in for several index parameters. These general factories require an `UpdaterMode` argument. The `ofStaticFieldUpdater`, `ofFieldUpdater`, and reference-array `ofArrayUpdater` conveniences instead fix `PUBLISH_ONCE` and return `Supplier`, `Function`, and `ObjIntFunction`, respectively. All updater modes reserve the selected location's Java default value to mean uninitialized and reject a computation that produces that value.

`Lazy.UpdaterMode`	**Access and coordination**	**Failure**	**Principal benefit and cost**
`PLAIN`	Plain read and write, without CAS or exclusion; each computing caller returns its own result	Propagated directly; retry remains possible	Least coordination, at the cost of possible overwrites, stale reads, and no safe-publication guarantee; plain `long`/`double` access may tear
`PUBLISH_ONCE`	Acquire observation and atomic release installation of a non-default value; competing candidates race and successful callers return the installed winner	Propagated directly; retry remains possible	No per-receiver coordination state, at the cost of duplicate work and discarded candidates
`SERIALIZED`	Acquire read, release write, and one computing thread at a time for the same field and receiver (or static field), or physical array element, across updater objects	Propagated directly without being retained; a later or waiting caller may retry	Avoids concurrent computations, at the cost of waiting and auxiliary coordination state

`SERIALIZED` shares coordination state across updater objects for a field and tracks instance receivers by weak identity. For arrays of any rank, its key is the identity of the final container array and the final index. The preceding indices only traverse outer dimensions. Two outer arrays or owner fields that refer to the same selected container array therefore coordinate access to the same element, as do direct-array and field-backed updaters that reach it. If a computation throws or returns the reserved default value, the location remains uninitialized; the next caller to acquire that state may compute again. The state must remain shared after failure so waiting and newly arriving callers cannot compute concurrently. After success, the location retains the result and the coordination state may be discarded. Cleared receiver or container-array entries are removed on a later uninitialized access, so their metadata may remain until then or until class unloading.

Updaters sharing a physical storage location must all use the same mode and should have interchangeable computing functions. For a use-site updater, this also applies to callbacks supplied on different calls to the same updater. The location must not be reset or written directly while they are in use; compatible same-mode updaters may share it. `PLAIN` is appropriate only when callers accept its weaker outcomes or provide their own synchronization. It is available only to updater factories, where the caller explicitly controls storage, without weakening the guarantees of cache factories in the same class.

Failure policy: preserve the Error category without retaining the original failure

The discussion explicitly chose to propagate an initial `Error` directly and keep its failure permanent in `COMPUTE_ONCE`. Subsequent observations remain in the `Error` hierarchy, using the dedicated `Lazy.InitializationError` rather than `NoSuchElementException`.

Computation outcome	**Access executing the computation in** `COMPUTE_ONCE`	**Access observing its recorded failure in** `COMPUTE_ONCE`	**Failing access in** `PUBLISH_ONCE`
Throws an `Error`	The same error, unchanged	A new `InitializationError`, no cause; original error class name when available	The same error, unchanged; retry remains possible

Computation outcome	Access executing the computation in `COMPUTE_ONCE`	Access observing its recorded failure in `COMPUTE_ONCE`	Failing access in `PUBLISH_ONCE`
Throws another throwable	`NoSuchElementException` with that throwable as cause	`NoSuchElementException`, no cause; original throwable class name	The original throwable; retry remains possible
Returns null	Computation failure represented by a `NullPointerException`, wrapped as above	The recorded permanent failure	Direct `NullPointerException`; retry remains possible

Waiting callers count as subsequent accesses. The failure state does not retain the original throwable. A later `InitializationError` reports an earlier failure, not a fresh occurrence of the original error condition.

The benefit over wrapping everything in `NoSuchElementException` is that ordinary `catch (Exception)` handling does not accidentally absorb subsequent observations of an `Error`. Keeping only failure information also limits retention of exception stacks, application objects, and class loaders through the failure state. The costs are a new public error type and reduced diagnostics on later access: the original cause, stack, and exact error subtype are no longer available there. `NoSuchElementException` also has a second meaning in traversal, so a lazy failure must not be interpreted as end-of-iteration.

The Javadoc cites JLS 12.4.2. The analogy is deliberately limited: class initialization propagates an initial `Error` unchanged and later reports erroneous initialization with `NoClassDefFoundError`; `Lazy` uses its own failure types and storage rules. It does not import the whole class-initialization protocol.

Same-thread recursive access to an uninitialized `COMPUTE_ONCE` slot throws `IllegalStateException`. The outer computation may catch it and succeed; it becomes sticky only if it escapes that computation. Recursive `PUBLISH_ONCE` computation need not be detected. Cross-thread dependencies can deadlock in `COMPUTE_ONCE`. No general cycle detector was added.

In `PUBLISH_ONCE`, a failing computation still throws even if another thread has installed a value in the meantime. Substituting the winner for that failure would hide an outcome of user code; the chosen rule preserves exception propagation at the cost of some accesses failing after storage has become successfully initialized.

Memory consistency: publication, without repeated volatile synchronization

The adopted cache guarantee is release publication and acquire observation, for both successful values and permanent `COMPUTE_ONCE` failures. Actions before publication happen-before actions after an access observing that publication, including actions in a handler for a recorded failure. The updater's `PUBLISH_ONCE` and `SERIALIZED` modes publish successful values with release effects and observe them with acquire effects. `PLAIN` makes no such publication guarantee, and `SERIALIZED` does not retain failures.

Alternative	Benefit	Cost / status
Volatile-access contract	Stronger ordering, including a total order of volatile accesses	Stronger than the selected publication requirement; adds obligations to repeated access (e.g., prevents @Stable access). Replaced in this branch.
Acquire/release publication	Preserves visibility of initialization while allowing later reads to be elided when publication is preserved	Does not guarantee additional ordering or visibility for subsequent mutations, or total ordering across different lazy values. Adopted.
Plain access without publication guarantees	Potentially weaker access costs	Requires a weaker user model or external synchronization. Excluded from cache factories; available as `Lazy.UpdaterMode.PLAIN` for field and array-element updates.

The distinction for discarded computations was corrected during review:

- A successful access whose own candidate loses **does acquire the winner's publication**.
- That does **not publish the losing computation's own actions to other readers** through the lazy slot.
- Actions performed by a failed `PUBLISH_ONCE` computation are likewise not guaranteed to be published to other threads through that slot.

An atomic operation is not automatically sufficient for both directions. In particular, `compareAndExchangeRelease` reads its witness with plain semantics. The updated implementations acquire-read the installed value after a losing release exchange. This is an implementation technique satisfying the contract, not an instruction sequence required of every implementation.

The revised contract permits constant folding and elision of later value or failure reads only while preserving the applicable publication guarantee. It does not promise folding on every VM, equate all final-field chains with trusted fields, or make subsequent mutation of a returned object thread-safe. Lazily obtaining an array also does not make its elements lazy. Internal volatile operations remain permissible; changing the public contract did not mean removing every volatile field from the implementation.

Caller-owned field and array-element storage: the updater family

Brian's email identified an allocation-sensitive case: many cheap, short-lived objects whose caches are usually never read. Using a field already in each receiver avoids allocating a lazy holder per receiver. The earlier named-field `Lazy.ofField` overload explored that approach; the current `Lazy` class exposes caller-owned fields and array elements through its updater factories. The general factories' `Class<T>` argument identifies the functional interface implemented by the returned updater; it is not a token for the stored value type. A declaration-site general factory also requires the computing function to implement `T`. A use-site factory instead requires the updater's first method parameter to be a functional interface for the computation. The convenience factories accept an owner or array element type and return familiar or narrowly shaped functional interfaces without requiring a separate interface declaration.

The adopted updater factories have these shapes (with Javadoc omitted):

```
static <T> Supplier<T> ofStaticFieldUpdater(Class<?> owner, String name,
                                            Supplier<T> computing);
static <R, T> Function<R, T> ofFieldUpdater(Class<R> owner, String name,
                                            Function<R, T> computing);

static <T> T ofLocationUpdater(UpdaterMode mode, Lookup lookup, Class<T> type,
                               String fieldSelector, T computingFunction);
static <T> T ofLocationUpdater(UpdaterMode mode, Lookup lookup, Class<T> type,
                               String fieldSelector);

static <T> T ofArrayUpdater(UpdaterMode mode, Class<T> type, T computingFunction);
static <T> T ofArrayUpdater(UpdaterMode mode, Class<T> type);
static <T> ObjIntFunction<T[], T> ofArrayUpdater(
        Class<T> elementType, ObjIntFunction<T[], T> computing);
```

The four general `ofLocationUpdater` and `ofArrayUpdater` overloads require `UpdaterMode`, keeping their coordination choice visible. `PUBLISH_ONCE` is listed first in `Lazy.UpdaterMode` and is the usual choice when concurrent duplicate computations and discarded candidates are acceptable. Enum order helps discovery but does not supply a Java default. The three narrower conveniences deliberately fix `PUBLISH_ONCE`; callers needing another mode or another coordinate shape use a general factory. This reduces arguments for common static-field, instance-field, and one-index reference-array updates while preserving an explicit policy choice for the general shapes. Removing the former general no-mode overloads still breaks sources that used them.

The two field conveniences are caller-sensitive. Each identifies the caller class and obtains a full-privilege lookup for that class through the internal `JavaLangInvokeAccess` bridge. If `lookup.findClass(owner.getName())` resolves to the supplied `owner`, the factory retains that lookup. If resolution finds a different class or throws `ClassNotFoundException` or `IllegalAccessException`, it instead uses `lookup.in(owner)`. This conditional scope keeps the caller's private, package, and protected access where the caller's loader already resolves the

intended class, while anchoring resolution in `owner`'s loader when the two differ. The fallback may drop lookup modes but cannot add privileges. It lets an accessible public owner in a custom loader be selected without treating a same-named class in the caller's loader as that owner. An unrelated private field remains inaccessible.

`ofStaticFieldUpdater` rejects an empty `name` or one containing a dot, then adapts `computing` to the non-generic `StaticObjectFieldSupplier` and delegates to `ofLocationUpdater(UpdaterMode.PUBLISH_ONCE, selectedLookup, StaticObjectFieldSupplier.class, owner.getName() + "." + name, adaptedComputing)`. The selected field must be static and mutable. The returned supplier reads it on `get()` and invokes the retained computation only while it has its reserved default value. `ofFieldUpdater` uses a simple name to select a mutable instance field of `owner`; its returned `Function<R,T>` receives a different `R` on each `apply` call and uses the field in that receiver as the storage location. As with the static convenience, field access is checked when the updater is created, but an erased or broader return type cannot prove every value conversion at compile time.

The static convenience still resolves the owner's name inside the general selector. A hidden class cannot be named that way, so its static fields are outside this convenience's scope. The instance convenience passes the owner `Class<R>` directly to storage selection and does not have that particular restriction.

The `ofArrayUpdater(Class<T>, ObjIntFunction<T[],T>)` convenience selects one element of a directly supplied reference array. Here `Class<T>` denotes the array's element type, unlike the general factory's `Class<T>` interface token. It rejects primitive and `void` element-type tokens at construction. An array class such as `String[].class` is a valid reference element type, so this form can select a row from a multidimensional array, but it always consumes exactly one index. Its returned function takes the array and index on each `apply` call and may therefore serve multiple arrays. Primitive arrays, multiple index coordinates, and other custom array shapes remain available through the general mode-taking factory. The `ObjIntFunction` name follows the `ObjIntConsumer` convention in `java.util.function`: one reference argument, then one `int`, with a reference result. Its type variables put input `T` before result `R`, and its functional method is `apply(T t, int value)`. The `value` is specifically an array index in this convenience.

The public caller-sensitive methods call `Reflection.getCallerClass()` directly. They do not expose a separate adapter taking a caller-class argument, because deep reflection could invoke such an adapter with a forged class. A focused run of the static convenience confirmed that direct calls can update the caller's own or a nestmate's private static field, while an unrelated caller's access to a private field is denied. In the same run, `Method.invoke` called from the nest and a method handle created with that nest's lookup both reached its private static field. The tests also confirmed that equal binary names in separate custom loaders select distinct public fields. These results illustrate that the invocation context matters; a wrapper calling a convenience uses the wrapper's access, while a `Lookup` passed to `ofLocationUpdater` is an explicit capability that a caller can delegate. Access is checked when an updater is created; the returned function may then be passed to other code without another caller access check on `get()` or `apply()`.

A method handle for a caller-sensitive field convenience may invoke it through a VM-generated nestmate of the class that created the handle. That preserves private nest access, but the generated class is not necessarily a subclass of another owner. A one-off probe on the static convenience confirmed that a direct call from a subclass could select a protected static superclass field in another package, while a method handle created by that subclass was denied access to the same field. The general factory with an explicit `Lookup` serves that case. An exact-caller reflective adapter could recover the subclass identity, but would expose a private method taking a caller-class argument that deep reflection could forge; this branch does not make that tradeoff.

The conveniences save an accessor-interface declaration and mode argument, and the field variants also save an explicit lookup. None offers `PLAIN` or `SERIALIZED`; those uses require a general factory. `Supplier<T>` and `Function<R,T>` do not establish the selected field's value type at compile time: the result type is erased, so a mismatched computation or preinitialized value can fail at access time. A primitive field's result is boxed through the returned standard interface. The array variant is statically limited to one index and reference elements; the general factory offers more shapes but requires a concrete functional-interface token.

Convenience	Benefit	Cost or boundary
`ofStaticFieldUpdater`	A `Supplier<T>` for one static field, without a lookup or custom accessor interface	Caller-sensitive access, erased value type, and one fixed mode

Convenience	Benefit	Cost or boundary
`ofFieldUpdater`	A reusable `Function<R,T>` that stores the result separately in each receiver's field, without allocating an updater per receiver	Caller-sensitive access, erased value type, and one fixed mode; `Class<R>` is needed to identify the receiver's field
`ofArrayUpdater(Class<T>, ObjIntFunction<T[],T>)`	One updater can serve many arrays; `Class<T>` fixes the element type and the call takes an ordinary array plus index	Adds a new public functional interface and supports only reference elements and one index coordinate; this prototype also adds wrapper calls and runtime type checks

For example, these three shapes can coexist in one class:

```
import java.lang.util.Lazy;
import java.util.function.Function;
import java.util.function.Supplier;

final class Example {
    private static String shared;
    private String derived;

    static final Supplier<String> SHARED =
            Lazy.ofStaticFieldUpdater(Example.class, "shared", () -> "ready");
    static final Function<Example, String> DERIVED =
            Lazy.ofFieldUpdater(Example.class, "derived", e -> "value");
    static final Lazy.ObjIntFunction<String[], String> ELEMENT =
            Lazy.ofArrayUpdater(String.class, (array, index) -> "item " + index);
}
```

The function returned for `derived` can be applied to multiple `Example` instances, while `ELEMENT` can be applied to multiple `String[]` arrays. Each receiver field or array element is a separate storage location.

Annotating the merged `Lazy` class with `@PreviewFeature` makes every factory a preview entry point. Nested public types keep their preview annotations. Previously the updater no-mode overloads were unannotated while the mode-taking forms were preview. Requiring preview opt-in for all factory calls is intentional while this API's surface and contract are being shaped.

An overload with a `computingFunction` argument binds one computation when the updater is created; one without it takes a computation at each access. In the latter form the updater's first method parameter is a functional interface whose single abstract method accepts the updater's remaining storage arguments in order and produces a compatible value. The factory checks both method shapes and the selected storage type. A callback may return a subtype of reference storage or use boxing, unboxing, or widening primitive conversion; a broad reference result is checked when computed. The callback parameter is not a storage coordinate. `ofLocationUpdater` names a field, optionally indexing into an array held by that field; `ofArrayUpdater` operates on an array supplied as the first storage argument. `ofArrayUpdater` is deliberately array-specific. An ordinary receiver type does not identify which of its fields to update without a selector. Other intrinsic storage carriers, such as atomic holders, would need their own explicit contract if added later.

Storage selected	Selector and storage-argument shape	Consequence
Instance field	Simple field name, such as `"radius"`; receiver followed by no indices	The receiver supplies storage for each invocation; a concrete SAM receiver type or fixed generic specialization identifies its owner
Static field	Binary owner name, dot, and field name, such as `"example.Answer.value"`; no storage arguments	The named class supplies one shared location; the static use-site interfaces cover `int`, `long`, and reference values
Element reached through an instance array field	Simple array-field name; concrete owner followed by one or more `int` indices, up to array rank	The element, not the array-reference field, is the storage location

Storage selected	Selector and storage-argument shape	Consequence
Element reached through a static array field	Qualified static array-field name; one or more `int` indices, up to array rank, and no receiver	The selected element is the storage location; standard `IntUnaryOperator` or `IntBinaryOperator` can fit one- or two-index declaration-site cases
Element of a direct array	No selector; concrete array followed by one or more `int` indices, up to array rank	The updater may serve many arrays of the same type

For use-site computation, prepend one callback parameter to the storage arguments in this table. The callback receives those same storage arguments. The named field or selected array element remains the location whose default value signals that computation is needed.

Nine pre-canned accessor interfaces in `Lazy` cover common use-site shapes. The six primitive interfaces retain their fixed `int` or `long` results: `IntArrayAccessor` and `LongArrayAccessor` access one-dimensional arrays, `IntFieldAccessor<R>` and `LongFieldAccessor<R>` access instance fields, and `StaticIntFieldAccessor` and `StaticLongFieldAccessor` access static fields. The three reference-valued counterparts are `ReferenceArrayAccessor<E>` for an element of a directly supplied `E[]`, `ReferenceFieldAccessor<R,V>` for an instance field, and `StaticReferenceFieldAccessor<V>` for a static field. Each accessor's outer method is named `getOrCompute`; the nested array `Computer` methods remain `apply`. Their callbacks likewise receive the storage coordinates, if any, and run only when the selected location has its reserved default value. A custom callback-first interface remains available for other primitive types or index shapes.

Unlike the non-generic primitive array and static-field accessors, each reference accessor needs a concrete, non-generic subinterface as its factory `Class` token, for example `interface Names extends ReferenceArrayAccessor<String> { }`, `interface Title extends ReferenceFieldAccessor<Document,String> { }`, or `interface Caption extends StaticReferenceFieldAccessor<String> { }`. This fixes the array element type, receiver and value type, or static value type that a raw generic token cannot supply. The selected reference field's existing value must be compatible with the accessor result; a newly computed value must also be storable in that field. A parameterized accessor value type must exactly match the field's declared generic type; the factory does not resolve type variables inherited through a generic field owner. A raw or broader result is checked against the field's erased storage type before publication, but Java cannot check its generic arguments at runtime. As with other reference locations, `null` is reserved for the uninitialized state and cannot be a successful computed result.

The field selector names one field; it is not a Java expression or a path through several fields. For a nested static owner, the selector uses its binary name, such as `example.Outer$Inner.value`. A field declared directly by the named owner or receiver type takes precedence. Otherwise, the nearest superclass declaration is selected if no interface declares the same name; with no superclass declaration, exactly one interface declaration is accepted. A superclass/interface conflict or multiple interface declarations are rejected as ambiguous. The search considers declarations even when a field is not inherited in Java source; the supplied lookup must still authorize access. A static caller can name the declaring class or interface to resolve such a conflict. Field access is authorized through the named owner or receiver type, allowing an accessible public subclass to expose an inherited public field from an inaccessible superclass. The `Lookup` resolves a static owner name using its lookup class's loader and authorizes access. Thus identical selector text can designate different classes under different loaders. Both field conveniences instead take an owner `Class` token, making that field's loader identity explicit at the call site and deriving lookup authority from the caller.

For an instance field, the receiver is normally inferred from the updater interface's **concrete erased receiver parameter type**. `Function<Point, Double>` does not provide this type to the factory: its `apply` method erases the parameter to `Object`. More generally, a raw generic interface token such as `Function.class` or `Supplier.class` is rejected at factory creation with an `IllegalArgumentException` explaining the erased type variable and how to declare a non-generic interface with concrete method types. Merely extending `Function<Point, Double>` still inherits an `Object` receiver; a subtype can instead explicitly redeclare `Double apply(Point)` so javac supplies a default bridge, though that saves little over a dedicated interface. The pre-canned generic interfaces are a deliberate convenience: a non-generic subinterface such as `CounterAccessor extends IntFieldAccessor<Counter>` fixes the owner without redeclaring a method, while the reference forms above fix their element or field-value types as well. The raw `IntFieldAccessor.class`, `LongFieldAccessor.class`, and reference-accessor base tokens leave

required types unresolved and receive a diagnostic showing the necessary specialization. The field name is validated against the inferred receiver type, and the selected field type is checked against the updater's result type. A static field needs no receiver, so a non-generic zero-argument interface such as `IntSupplier` remains useful for declaration-site computation.

```
class Point {
    private final double x, y;
    private Double radius;                // null means not initialized

    Point(double x, double y) { this.x = x; this.y = y; }
    interface Radius { Double getOrCompute(Point point); }

    private static final Radius LAZY_RADIUS = Lazy.ofLocationUpdater(
            Lazy.UpdaterMode.PUBLISH_ONCE, MethodHandles.lookup(),
            Radius.class, "radius",
            point -> Math.hypot(point.x, point.y));

    double radius() { return LAZY_RADIUS.getOrCompute(this); }
}
```

The boxed `Double` permits a computed positive zero: `null`, rather than `+0.0`, is the reserved uninitialized value. For a primitive `double` field, positive zero could not be stored as a computed result.

The pre-canned instance accessor gives the computation its own interface. A concrete subinterface fixes the receiver type, while each access supplies a callback before that receiver:

```
class Counter {
    private int seed = 41;
    private int cached;                   // zero means not initialized

    interface CounterAccessor extends Lazy.IntFieldAccessor<Counter> { }

    private static final CounterAccessor LAZY = Lazy.ofLocationUpdater(
            Lazy.UpdaterMode.SERIALIZED, MethodHandles.lookup(),
            CounterAccessor.class, "cached");

    int value() { return LAZY.getOrCompute(counter -> counter.seed + 1, this); }
}
```

Here `LAZY` retains no computation. The lambda has a `Counter` parameter and runs only when `cached` is zero. In `SERIALIZED` mode, the first successful callback for a location supplies its value; a later access receives that value without invoking its own callback. After a callback fails, a later access may try its own callback. A caller can instead declare a custom updater interface with its own `Computer` and access method for an instance field of another value type.

A static selector includes the owner's binary name. An index-only interface can reach an element through a static array field without passing the array at each invocation (imports omitted):

```
package example;

class Answer {
    private static int value;
    private static final IntSupplier LAZY_VALUE = Lazy.ofLocationUpdater(
            Lazy.UpdaterMode.PUBLISH_ONCE, MethodHandles.lookup(),
            IntSupplier.class,
            "example.Answer.value", () -> 42);
}

class Holder {
    private static final int[] ints = new int[8];
    private static final int[][] matrix = { new int[4], new int[4] };
    private static final int[][][] volume = new int[2][3][4];

    interface Voxel { int getOrCompute(int plane, int row, int column); }

    private static final IntUnaryOperator LAZY_INTS = Lazy.ofLocationUpdater(
            Lazy.UpdaterMode.PUBLISH_ONCE, MethodHandles.lookup(),
            IntUnaryOperator.class,
            "example.Holder.ints", index -> index + 1);

    private static final IntBinaryOperator LAZY_CELLS = Lazy.ofLocationUpdater(
            Lazy.UpdaterMode.SERIALIZED, MethodHandles.lookup(),
            IntBinaryOperator.class, "example.Holder.matrix",
            (row, column) -> 1 + row * 4 + column);

    private static final Voxel LAZY_VOXELS = Lazy.ofLocationUpdater(
            Lazy.UpdaterMode.PUBLISH_ONCE, MethodHandles.lookup(),
            Voxel.class, "example.Holder.volume",
            (plane, row, column) -> 1 + plane * 12 + row * 4 + column);
}
```

`LAZY_INTS.applyAsInt(3)` selects `ints[3]`; `LAZY_CELLS.applyAsInt(1, 2)` selects `matrix[1][2]`; `LAZY_VOXELS.getOrCompute(1, 2, 3)` selects `volume[1][2][3]`. The `final` static fields are paths to mutable elements; the fields themselves are not updated. For any rank, index arguments follow array dimensions from outermost to innermost. There is no accessor expression in which to swap, repeat, or transform them.

The pre-canned static field accessors accept `IntSupplier`, `LongSupplier`, or `Supplier<V>` callbacks for `int`, `long`, or reference fields. These callbacks take no parameters because there is no receiver or index to pass through:

```
class UseSiteAnswer {
    private static int value;

    private static final Lazy.StaticIntFieldAccessor ACCESS =
            Lazy.ofLocationUpdater(Lazy.UpdaterMode.PUBLISH_ONCE,
                    MethodHandles.lookup(),
                    Lazy.StaticIntFieldAccessor.class,
                    UseSiteAnswer.class.getName() + ".value");

    static int answer() { return ACCESS.getOrCompute(() -> 42); }
}
```

The earlier `Counter.CounterAccessor` example fixes the instance owner through its `IntFieldAccessor<Counter>` specialization. Supplying the raw generic base type instead leaves the owner unknown and fails at factory creation.

For an instance array field, `ofLocationUpdater` takes the field's simple name and the interface includes its owner. For a direct array, `ofArrayUpdater` takes no selector and the interface includes the array itself:

```
class Names {
    private final String[] entries = new String[8];
    interface Entry { String getOrCompute(Names owner, int index); }
    private static final Entry LAZY_ENTRY = Lazy.ofLocationUpdater(
            Lazy.UpdaterMode.PUBLISH_ONCE, MethodHandles.lookup(),
            Entry.class, "entries",
            (owner, index) -> "entry-" + index);
}

class Values {
    private final int[] entries = new int[8];
    private static final Lazy.IntArrayAccessor.Computer LAZY_ENTRY = Lazy.ofArrayUpdater(
            Lazy.UpdaterMode.PUBLISH_ONCE, Lazy.IntArrayAccessor.Computer.class,
            (array, index) -> index + 1);

    int entry(int index) { return LAZY_ENTRY.apply(entries, index); }
}
```

The use-site form makes the callback first and keeps the array and index together. A call with the built-in `int[]` accessor is:

```
int[] entries = new int[8];
Lazy.IntArrayAccessor access = Lazy.ofArrayUpdater(
        Lazy.UpdaterMode.PUBLISH_ONCE,
        Lazy.IntArrayAccessor.class);
int value = access.getOrCompute((array, index) -> array.length + index + 1,
                            entries, 0);
```

The callback receives the array and index, with no unused updater parameter. A cached nonzero element is returned without calling it. In `PUBLISH_ONCE` mode, callbacks can race and successful accesses return the installed winner; a losing candidate's result is discarded. A one-index reference array can use `ReferenceArrayAccessor<E>` through a concrete subinterface; other primitive element types or multiple indices can use a custom callback-first interface.

The direct-array shape also permits an array-valued element. For `int[][][]`, one index selects an `int[][]` plane, two select an `int[]` row, and three select an `int` cell:

```
class Volume {
    interface Plane { int[][] getOrCompute(int[][][] array, int plane); }
    interface Row { int[] getOrCompute(int[][][] array, int plane, int row); }
    interface Cell { int getOrCompute(int[][][] array, int plane, int row, int column); }

    private static final Plane PLANES = Lazy.ofArrayUpdater(
            Lazy.UpdaterMode.PUBLISH_ONCE, Plane.class,
            (array, plane) -> new int[3][]);
    private static final Row ROWS = Lazy.ofArrayUpdater(
            Lazy.UpdaterMode.PUBLISH_ONCE, Row.class,
            (array, plane, row) -> new int[4]);
    private static final Cell CELLS = Lazy.ofArrayUpdater(
            Lazy.UpdaterMode.PUBLISH_ONCE, Cell.class,
            (array, plane, row, column) ->
                    1 + plane * 12 + row * 4 + column);

    private final int[][][] volume = new int[2][][];

    int cell(int plane, int row, int column) {
        PLANES.getOrCompute(volume, plane);
        ROWS.getOrCompute(volume, plane, row);
        return CELLS.getOrCompute(volume, plane, row, column);
    }
}
```

On a new `Volume`, the outer calls install a plane and row into `null` slots before the cell updater traverses them. An intermediate array reference must be non-null for a deeper selection; a null reference or an invalid index fails before computing. Each updater selects a distinct physical element and uses that element's default value as its uninitialized

sentinel. If `CELLS` used `SERIALIZED` mode, the final container array and final index would determine coordination: the `int[]` row and `column`. All forms rely on a concrete array parameter in the interface. A generic interface erased to `Object` cannot reveal the direct array type merely through `Class<T>`.

For `ofLocationUpdater`, the lookup is an access capability, not a bypass: it authorizes the updater interface and selected field, and lookup-imposed receiver restrictions remain in force. The capability can be delegated through helpers, at the cost of an extra argument at each call. `ofStaticFieldUpdater` and `ofFieldUpdater` instead obtain caller-sensitive authority and keep the full caller lookup when it resolves the supplied owner exactly; otherwise they scope that lookup to `owner`, which may reduce access while preserving the class identity across loaders. This conditional selection cannot grant access that the caller lacked. It preserves ordinary caller access to private nestmate fields when the owner resolves in the caller's loader; the static case also preserves direct-call access to protected static fields. This follows the established lookup capability model. Direct `ofArrayUpdater` access needs no field resolution or caller-supplied lookup. Its array parameter and index parameters identify the element. The general factory checks its concrete updater interface; the convenience instead uses its element-type token. Adapter generation remains an internal implementation detail.

All updater entry points share the reserved-value storage protocol; the three conveniences use the `PUBLISH_ONCE` subset of the general factories' modes. The selected mutable field or physical array element uses its Java default value as the uninitialized sentinel: `null`, `false`, integral zero, or positive floating-point zero. A computation returning that value throws `IllegalArgumentException`, and a preexisting non-default value is returned without computation. A selected field cannot be `final`, but a final array-reference field can lead to mutable elements. A null array along a path or invalid index fails before computing. A null-restricted reference field is rejected at factory creation; a null-restricted reference array element cannot supply the required `null` sentinel and is rejected on access. An array-reference path is resolved once per invocation. The caller must safely publish any receiver, array, or preinitialized value and must not reset a selected location or change intermediate array references while its updater is in use.

The modes retain their distinct tradeoffs: `PLAIN` permits racy plain writes without safe publication, `PUBLISH_ONCE` uses acquire observation and atomic release publication to retain one non-default winner, and `SERIALIZED` runs at most one computation at a time per physical location, with retry after failure. Across either factory, aliases of the same selected container array and index designate one location and share `SERIALIZED` coordination. Updaters sharing a location must use the same mode and should have interchangeable computing functions, including callbacks provided to a use-site updater on separate calls. Neither factory eagerly adds a value holder to every receiver or array element; `SERIALIZED` may allocate auxiliary weakly keyed coordination state on access. Generic functional-interface parameterizations are erased at `Class<T>`, so some result compatibility cannot be proved at factory creation; an incompatible computed value may fail at a later cast or array store.

Path not taken: cracking accessor lambdas

An earlier revision supplied a second lambda to select storage, for example `point -> point.radius`, `index -> Holder.ints[index]`, or `(array, row, column) -> array[row][column]`. This made the selector type-checked by the Java compiler, supported IDE rename tools, and avoided string field names. It also let one factory, then called `ofLocation`, cover fields and direct arrays. The prototype inspected generated lambda implementation bytecode without executing the accessor.

That advantage came with a complex and fragile contract. The implementation had to recover the lambda's implementation method, inspect bytecode and constant-pool references, recognize array-load opcodes and method-handle forwarding shapes, and then recheck lookup authority. Captures, method references, transformed indices, and several compiler or hidden-class shapes needed exclusions or special cases; custom class loaders complicated access to implementation bytes. Even simple source expressions depended on the exact bytecode shape emitted by the compiler. The mechanism also allowed reversed index arguments, adding a semantic choice whose only expression was the selector lambda. Those costs outweighed the convenience of a type-checked field expression for this prototype.

Storage-selection choice	Pros	Cons / decision
Named field with `String` and `Lookup`	Simple selector grammar; no inspection of compiler-generated accessor code; explicit access authority	Field names are checked at factory creation rather than compilation, and static names resolve in the selected lookup's loader. Adopted for `ofLocationUpdater`; `ofStaticFieldUpdater` and `ofFieldUpdater` derive a caller lookup for their narrower shapes.
Direct array identified by a concrete SAM parameter	No selector string or storage-selection callback; coordinates have one order	Requires a separate `ofArrayUpdater` name and a concrete erased array parameter for the general factory. The one-index reference-array convenience instead takes an element-type token.
Cracked accessor lambda	Compiler type-checks the storage expression and IDE renames can update it	Depends on generated bytecode and rejects several otherwise ordinary lambda shapes. Dropped.

The current design uses a string only where a field genuinely needs a name and uses `ofArrayUpdater` where the concrete SAM storage parameters and indices already identify a direct array element. This removes accessor bytecode inspection from the public contract and implementation path. The computation remains a normal lambda or method reference; it may capture state. A declaration-site updater retains it, while a use-site updater receives it on each access. The design gives up compile-time checking and automatic refactoring of field names; a wrong selector is detected when the factory is created. A more general string-free overload for arbitrary receivers was not adopted because a receiver type alone cannot identify which of its fields is the desired storage location.

Collections: retain ordinary shapes, specify the lazy boundary

The collection factories create unmodifiable collections. Every factory invocation supplies independent initially uninitialized storage; this does not require distinct result objects if callbacks return the same object. Views and map entries share their originating storage. Empty-instance reuse is permitted when construction has size zero, no keys, or no candidates; this permission does not turn a nonempty-candidate set that later filters to empty into shared lazy state.

Shape	What is fixed or copied eagerly	What is computed lazily	Main benefit / cost
List	Size and index domain	One element per index	Useful indexed interface and `RandomAccess` views; element-inspecting operations can trigger work or fail
Map	Copied key set	One value per key	Keys and size can be used without evaluating values; ordinary value-oriented operations can evaluate them
Set	Copied candidate set	One membership decision per candidate, retaining both true and false	Supports lazy filtering with stable membership; size and hash require all membership statuses

`ofSetCache` was added in this branch beyond Brian's final sketch. It computes membership of existing candidates, not new element objects or an unbounded stream.

List sublists and reversed views share original slots and compute with the original indices. Looking up an absent map key or an object outside a set's candidate domain does not invoke its computing function. An out-of-range list access fails before computation.

Map computations receive the **retained key from construction**, even when queried with a different but equal object. Set predicates receive the retained candidate on the same basis. The copies are shallow: the key/candidate objects themselves are retained, not cloned. This removes dependence on which equal query object arrives first. Its cost is

that information present only on a query object is intentionally not an input to the cached computation. Input sets must use equality-compatible membership; changing key/candidate equality or hash codes while retained is outside the contract. Map and set iteration order is unspecified.

Copying a lazy input set can evaluate **the input's memberships** and propagate its failures. Construction does not thereby compute the output map's values or the output set's memberships. This distinction preserves snapshot semantics while making clear that accepting a lazy input is not a promise of a computation-free factory call.

The following stronger collection choices were made:

Decision	Pros	Cons
Null results are computation failures; null keys/candidates are rejected during construction	Clear distinction between uninitialized storage and a retained result	Absent computed values need an explicit representation such as `Optional`
Unsupported mutations always throw, even for no-ops, without invoking computing functions or supplied mutation callbacks	Predictable unmodifiability; `sort`, `replaceAll`, `removeIf`, and map updates cannot accidentally perform work	More overrides than permissive default methods; no “successful” empty mutation
Entries remain valid after traversal and share the original per-key storage	Entries can safely escape an iterator; later access observes the same result or permanent failure	Retaining one entry can retain the map and other unresolved state
Returned suppliers, functions, and collections are not serializable, including empty collections	Avoids defining serialization of callbacks, incomplete state, and sticky failures	Applications must choose their own materialized or domain-specific representation
Scalar `equals`, `hashCode`, and `toString` do not compute; collection content operations may compute	Keeps scalar diagnostics harmless while retaining ordinary collection meanings	Collection logging, hashing, equality, or copying can run user code and fail

Obtaining map entries or their keys does not itself compute mapped values. Reading an entry's value does; `Map.Entry.copyOf` therefore may compute or report a failure while producing its detached copy. Unmodifiable collections can still contain mutable values. In particular, a lazy map's structural hash must reflect current value hashes rather than be permanently cached merely because its storage locations never change.

Traversal and streams: failures do not silently remove data

When the traversal itself encounters a lazy computation failure, the pending value or candidate is not consumed. A failed list-iterator `next` or `previous` leaves its cursor unchanged. Set iterators may advance past candidates already known to be absent, but keep a failing membership pending. The rules cover bulk traversal, `hasNext`/`hasPrevious`, and spliterators; a failed source access in `tryAdvance` does not invoke its action. A split that fails because of lazy computation must preserve the original's remaining coverage.

The benefit is that retry in `PUBLISH_ONCE` cannot silently skip data, while `COMPUTE_ONCE` repeatedly reports the same slot's recorded failure. The cost is extra traversal state and implementations beyond ordinary defaults. This is not transactional rollback: earlier values, memberships, and failures remain retained, and elements already delivered remain consumed.

Failures thrown by a traversal action follow the ordinary interface's continuation rules, including when that action makes its own failing lazy access. Such a failure must not be mistaken for failure of the traversal's own source access. More generally, user callbacks outside the computing function do not themselves create an additional initialization failure; lazy accesses inside them retain their own rules.

Creating streams or spliterators and querying their metadata do not compute values or report stored computation failures. Source spliterators, including those produced by splitting, report `NONNULL` and `IMMUTABLE`; lists and views additionally report `ORDERED`, `SIZED`, and `SUBSIZED`. Map key/entry views report `DISTINCT`, `SIZED`, and `SUBSIZED`; value views report the latter two. Lazy sets report `DISTINCT` and need not report an exact size before it is known without evaluation.

The design does not promise minimal evaluation by every traversal, split, or parallel operation. Short-circuiting can compute more than the final result consumes. Conversely, a sized `count()` can avoid element computation altogether, even when some element has a recorded failure. This preserves stream optimization freedom. A terminal operation such as `count()` is not evidence that every lazy value has been evaluated; validation must explicitly traverse or access the relevant values. The current implementation's non-computing range splits are stronger than the minimum public guarantee and are not a new API commitment.

Execution, retention, and lifetime

Computations run synchronously on the accessing thread. `COMPUTE_ONCE` coordinates one slot; a shared collection callback may still execute concurrently for different slots. Waiting is uninterruptible and preserves an interrupt already present or received during the wait, on normal or abrupt completion. A computing function's own interruption handling is not constrained by that waiting rule. Cancellation, timeouts, and asynchronous execution are not provided.

This keeps access and exception propagation direct and avoids an executor policy. It also means that a blocking computation blocks its caller, and potentially other callers waiting for its result. Neither mode provides a general liveness guarantee.

Cache factories strongly retain the computing function while any slot remains unresolved, and release the invocation reference when all slots have either a result or permanent failure. Mixed successful and failed slots count as terminal. A view's lifetime is assessed against the whole original storage, including slots outside that view. Declaration-site `Lazy.ofStaticFieldUpdater`, `ofFieldUpdater`, `ofLocationUpdater`, and `ofArrayUpdater` updaters retain their computing functions for their lifetimes, even when the static-field updater selects one field that has been initialized. Instance-field and array updaters can encounter new receivers or elements later. Use-site updaters do not retain a computation between accesses; a callback supplied to an access can still be held while that invocation is in progress.

This rule bounds retention after completed initialization while ensuring that unresolved slots remain computable. It cannot guarantee garbage collection: values, keys, candidates, in-flight computations, and other application references may still retain the callback. `Lazy.ofScalarCache(CacheMode, Function)` does not otherwise retain its context, and the updater factories do not otherwise retain receivers or arrays; callbacks or results can themselves contain references. The updater's `SERIALIZED` mode uses weak receiver or selected-container-array keys for its auxiliary coordination state.

Records and Valhalla: supported patterns and limits

The discussion tested the derived-property use case for `record Point(int x, int y)`. That exact record cannot declare extra instance fields for `radius` and `phi`; see JLS 8.10.2. A static `ofScalarCache(CacheMode, Function)` would incorrectly share one slot across points. An ordinary class can declare per-instance holder or updater storage instead. Adding holders as record components would change its representation and generated equality; no transparent record cache facility or new language feature was implemented.

For the Valhalla value-class model examined here, a final field can reference a mutable lazy holder, and `ofScalarCache(CacheMode, Function)` with a noncapturing callback avoids capturing incompletely initialized `this`. A value-class instance field cannot itself serve as mutable storage for `Lazy.ofLocationUpdater` because value-class instance fields are final. A final field referencing a mutable array could in principle supply lazy array elements through `ofLocationUpdater` or pass that array directly to `ofArrayUpdater`, subject to the value-class and array rules; this combination has not been validated here. The important cost is equality: holder or array references participate in value-object `==` and inherited `Object.equals`, so objects with otherwise equal application data may compare unequal. Custom `equals`/`hashCode` can define logical equality but cannot change `==`. These points concern the Valhalla model, whose draft specification remains a separate language-development effort.

The benefit is a workable library pattern without adding language machinery. The costs are holder storage and possible equality consequences. The API does not make all capturing lambdas illegal, guarantee allocation-free value-class caching, or supply an indexed-relative collection shape whose ordinary `List.get` accepts an owning value object.

Alternatives outside the adopted scope

The distinctions in this table matter: absence from the current surface is not proof of permanent rejection.

Alternative	Status in this work	Reason or unresolved tradeoff
Lazy fields / cached methods as new language features	Discussed in Brian's background; not implemented here	Static and instance use cases do not divide cleanly; records and value classes add separate constraints
Public stability annotation or general stable VarHandle/MethodHandle toolkit	Not exposed by this API	The high-level entry point targets common usage; possible lower-level facilities remain separate work
Full coordinate family beyond array indices	Present in earlier sketches; the updater layer supports any feasible array rank, optionally through one owner field or a static array field, but no general indexed-relative coordinate type	A smaller API loses non-array context-plus-index use cases and arbitrary coordinate transformations
Context class-token overload	Explored, not adopted	Better inference cues versus extra signature surface
Further primitive specializations	Pre-canned use-site `int` and `long` accessors cover one-dimensional arrays, instance fields, and static fields; reference counterparts now cover those same shapes, but other primitive types are not provided here	Custom updater and callback interfaces remain available, while a larger built-in family would increase the API surface
Null as a successful lazy result	Not offered	Non-null retained values simplify the contract; absence needs another representation
Reset, invalidation, eviction, or per-argument memoization	Not provided	These require a lifecycle beyond one retained result per storage location
Explicit initialization-state query or non-computing peek	Not provided	Standard return interfaces expose access, not a new lifecycle protocol
Serialized retry in cache factories	Not provided; `Lazy.UpdaterMode.SERIALIZED` offers it for field and array-element storage	Requires waiting and auxiliary coordination state
Independent policy flags	Not provided	The design deliberately stops short of the full policy/storage product
`COMPUTE_ONCE` for caller-owned updaters	Not provided by `Lazy.UpdaterMode`	At-most-once computation with permanent failure would require sticky failure state in addition to the selected field or array element
Segment-backed `Lazy.ofSegment`	Experimental overloads removed from the earlier `Lazy` prototype	A segment-backed factory would need its own storage and lifetime contract
Cancellation, timeouts, executor-backed evaluation	Not provided	No scheduling or task-management policy is added to lazy access
Removal or deprecation of `LazyConstant` and collection `ofLazy` factories	Not performed	Migration and compatibility policy for those earlier APIs require a separate platform decision

Implementation and validation status

The implementation work followed the specification reviews. It updated scalar, field, and collection publication; retained stable storage opportunities; added failure-preserving traversal and views; preserved original storage through map entries and key views; corrected equality exception propagation; and preserved publication through cached set

size/hash results. Existing legacy factory behavior was kept on its existing paths.

The source layout has one public facade, `java.lang.util.Lazy`, exported by `java.base`. It validates arguments and delegates to internal implementations. Scalar and collection behavior lives in `jdk.internal.util` (`LazySupport` and `LazyCollections`); field selection, updater coordination, and functional-interface adapters live in `jdk.internal.invoke` (`LazyUpdaterSupport` and `LazyUpdaterFactory`). The existing `List.ofLazy`, `Map.ofLazy`, and `Set.ofLazy` factories also use the internal collection implementation. Generated hidden classes may still implement updater interfaces; that adapter mechanism is separate from the dropped accessor-bytecode inspection. The caller-supplied lookup for `ofLocationUpdater` governs access to the selected field; `ofStaticFieldUpdater` and `ofFieldUpdater` derive a caller lookup through the internal access bridge, retaining it when it resolves the supplied owner and otherwise scoping it to that owner. `ofArrayUpdater` has no caller-supplied lookup. One- and two-index updaters retain specialized call paths. Higher-rank updaters collect their indices into an `int[]` before traversal, with a potential per-access allocation cost.

The historical validation results below record earlier revisions under their then-current class names and test paths. They describe what was checked at each stage, not validation of the merged `java.lang.util.Lazy` API. Compilation, Javadoc, and focused regression checks for the class merge are recorded after those earlier results.

The earlier validation sequence built `java.base` and passed 13 targeted jtreg test descriptions, comprising 821 successful JUnit invocations plus non-JUnit checks. Coverage included the new Lazy suites, legacy lazy-list/map/set tests, a negative compilation test, lookup/module tests, and MOAT collection checks extended to both coordinated modes. One new test file initially needed a generic assertion typing correction and then passed on a focused rerun; the combined results were all passing. Those recorded results predate the mode-interface and three-mode updater extension and do not validate that extension.

The preceding mode-interface variant was validated against the built JDK: the `java.base` and JDK API Javadoc builds passed; all 12 Lazy jtreg descriptions passed (376 JUnit executions), including compile-fail checks for the updater-only mode; and both `LazyUpdaters` jtreg descriptions passed (82 JUnit executions across normal and interpreter runs). The earlier MOAT run also passed; it was not repeated for that variant. Those results do not validate the independent enums or `SERIALIZED` retry behavior introduced by this revision.

The internal-package migration passed a clean JDK `images` build. On that build, 17 focused jtreg descriptions passed (904 JUnit executions), covering the independent modes, updater retry and field access, lazy factories and collections, and legacy list/map/set behavior. The throwable class-unloading and MOAT collection checks also passed as two additional jtreg descriptions.

After removal of the `Lazy.ofField` and `Lazy.ofSegment` overloads and their tests, and the updater-mode renaming, the pre-array revision passed the `java.base-java` and JDK API Javadoc builds. All 11 targeted jtreg descriptions passed: eight under `java/util/Lazy` and three under `java/lang/invoke/LazyUpdaters`, including 408 JUnit invocations. These runs establish the tested behavior, not a proof of all memory-model executions, measured allocation savings, or a performance comparison across VMs. The outstanding platform decisions remain adoption and release sequencing, the relationship to the earlier APIs, and whether workload evidence warrants any additional shapes or specializations.

For the earlier lambda-selector array-element extension, the `java.base-java` and JDK API Javadoc builds passed. All three `LazyUpdaters` jtreg descriptions passed, including 104 JUnit executions across normal and interpreter runs. Those tests covered one- and two-index direct and owner-field accessors, swapped indices, every primitive array-load opcode and reference elements, the three updater modes, aliasing of inner arrays, reserved-value retries, bounds and null failures, and lookup checks. The `java/util/Lazy` tests were not rerun for that extension, which did not change the `Lazy` implementation.

In that earlier lambda-selector design, index-only static-array access was folded into `ofLocation` and an earlier, different `ofArray` factory was removed. The updated JDK image and JDK API Javadoc build passed. The three focused `LazyUpdaters` jtreg descriptions (`OfStaticArrayLocationTest`, `OfLocationTest`, and `FindVarHandleTest`) passed with 130 JUnit invocations across normal and interpreter runs. They covered static one- and two-index accessors, reversed indices, the default and explicit modes, lookup checks, and the existing receiver-based forms. The later direct-array factory, now named `ofArrayUpdater`, has different parameters and semantics.

For the `LazyCaches` and `ofScalar` rename, the JDK image and JDK API Javadoc builds passed. All eight jtreg descriptions under `test/jdk/java/util/Lazy` passed (326 JUnit invocations), as did the MOAT collection test and the `LazyUpdaters` mode-mismatch compile-fail test. The test directory retained its historical `Lazy` name; at that stage, its source exercised `LazyCaches`. These results also precede the current selector redesign.

For the initial string-selector and direct-array design, before `ofArray` dropped its lookup parameter, the `java.base-java`, full JDK `images`, and JDK API Javadoc builds passed. All four focused `LazyUpdaters` jtreg descriptions and the updated `LazyDemo` passed, including 101 JUnit invocations across normal and interpreter runs and the mode-mismatch compile-fail check. The tests cover named instance and static fields, direct and field-backed arrays, row-valued selections, natural index order, the three modes, lookup access, inherited fields reached through public subclasses and interfaces, ambiguous-name rejection, and distinct static fields with the same binary name in separate class loaders. An initial static-array test expected an access error from `publicLookup`, but that lookup could not load the application class; the test was corrected to use a restricted lookup in the same loader and passed on rerun. These results validate the focused behavior, not performance claims or all possible memory-model executions.

After removing the lookup parameter from `ofArray`, the `java.base-java` and JDK API Javadoc builds passed. All four focused `LazyUpdaters` jtreg descriptions passed, including 101 JUnit executions across normal and interpreter runs; a test-only naming cleanup was followed by another successful `OfLocationTest` run. At that stage, the suite covered a direct array with a private functional interface and a private element type, while retaining `ofLocation` lookup-access tests. These checks do not establish performance or all possible memory-model executions.

For the arbitrary-rank extension, the `java.base-java` and JDK API Javadoc builds passed. All four focused `LazyUpdaters` jtreg descriptions passed, including 115 JUnit executions across normal and interpreter runs. New tests cover three- and four-index direct access, three-index instance and static array fields, higher-rank partial indexing, aliases, failures before computation, retries after a reserved result, and rejection when the index count exceeds array rank. These are representative ranks, not an exhaustive test of every JVM-legal method arity or memory-model execution.

Before adding the reference pre-canned interfaces, the declaration-site and use-site design passed the JDK API Javadoc build and all five `LazyUpdaters` jtreg descriptions, including 153 JUnit executions across normal and interpreter runs. The use-site tests covered direct and field-backed arrays, static and instance fields, the six pre-canned `int` and `long` shapes, all three modes, shared physical-element coordination, generic receiver specialization, callback validation, reference subtype results, primitive widening, and diagnostics for erased generic interface tokens and storage coordinates. This validates the exercised behavior at that stage, not the subsequently added reference interfaces, performance, or every memory-model execution.

With the three reference pre-canned interfaces, the `java.base-java` and JDK API Javadoc builds passed. All five `LazyUpdaters` jtreg descriptions passed, including 167 JUnit executions across normal and interpreter runs. The added tests cover all three updater modes for direct reference arrays and static and instance reference fields, cached and reserved-null results, array covariance, raw generic-token diagnostics, incompatible field types, and matching or mismatching parameterized field types through direct and indirect specializations, including wildcard bounds. These tests do not establish performance or every memory-model execution.

After requiring an explicit updater `Mode` and moving `@PreviewFeature` to the `LazyUpdaters` class, the `java.base-java` and JDK API Javadoc builds passed. All five `LazyUpdaters` jtreg descriptions and `LazyDemo` passed, comprising 169 JUnit executions across normal and interpreter runs. A client compilation using the built JDK rejected `LazyUpdaters` without preview enabled and succeeded with `--enable-preview`. These checks validate the migrated call sites and preview boundary, not source or binary compatibility with the removed overloads.

For the final two-facade documentation reconciliation, the JDK API Javadoc build passed, followed by all five `LazyUpdaters` and eight `LazyCaches` jtreg descriptions (494 JUnit executions). All 13 examples in the two class Javadocs were compiled individually and exercised with the built preview JDK. A separate value-class probe confirmed the documented rejection timing for null-restricted reference fields and array elements in all three updater modes. These checks establish that those examples were usable and the focused regressions passed; they do not validate the later one-class rename or prove every memory-model execution or measure performance.

For this one-class move, the `java.base-java`, full JDK `images`, and JDK API Javadoc builds passed. Strict direct Javadoc validation and compilation of all 13 snippets in the merged `Lazy` class also passed. The refreshed image exports `java.lang.util` and contains `Lazy`, without either former public facade. All 24 targeted jtreg descriptions passed: eight cache tests (327 JUnit executions), five updater tests (167), the MOAT collection test, and ten legacy

`LazyConstant` tests (477). The two negative-compilation tests matched diagnostics for the intended `CacheMode`/`UpdaterMode` type errors. A client using both factory families compiled and ran with preview enabled; compilation without preview correctly rejected the API. These checks validate the moved surface and representative behavior, not performance or every memory-model execution.

For the static-field convenience (then named `ofFieldUpdater`, now `ofStaticFieldUpdater`), the `java.base-java`, JDK `images`, and JDK API Javadoc builds passed. The focused `OfLocationTest` passed with 102 JUnit executions across normal and interpreter runs, covering public and same-nest private fields, unrelated-private denial, a malformed field name, reflective and method-handle invocation, and two custom loaders holding classes with the same binary name. Separate one-off probes confirmed that a caller in a named module with a package not opened to `java.base` can initialize its own private static field, that an external caller cannot invoke the internal lookup bridge to obtain a forged full-privilege lookup, and that method-handle invocation can lose cross-package protected subclass access even when a direct call succeeds. These checks validate the exercised access paths, not every class-loader or caller-sensitive invocation arrangement.

For the present static rename, instance-field convenience, `ObjIntFunction`, and reference-array convenience, the `java.base-java` and JDK API Javadoc builds passed. The focused `OfLocationTest` passed with 118 JUnit executions across normal and interpreter runs. New cases cover per-receiver and per-element storage, private caller fields, primitive field boxing, preexisting values, reserved-value retries, private reference components, array-valued elements, bounds, and rejection of primitive or `void` element-type tokens. Existing static tests also exercised the renamed entry point through direct, reflective, and method-handle invocation. These checks do not establish performance or every possible class-loader, caller-sensitive, or memory-model execution.